OpenZeppelin | security

GnoSwap Extended Audit

GnoSwap

December 23, 2025

Table of Contents

Table of Contents 2

Summary 4

Scope 5

System Overview 7
- Upgradeability Architecture 8
- Swap Callback 8

Security Model and Trust Assumptions 9
- Upgrade Authorities 9
- Upgrade to the Same Version 9
- Permission Reset During Implementation Changes 10
- Router Fee 10
- Privileged Roles 10

Critical Severity 12
- C-01 Router Swap Callback Caller Is Not Verified 12
- C-02 Swap Reentrancy Lock Is Ineffective 12

High Severity 13
- H-01 Incorrect Iteration Control in IsExternallyIncentivizedPool Stops Staking 13

Medium Severity 14
- M-01 None Permission Can Be Escalated to Read After Upgrade 14
- M-02 Inadequate Validation of Community Pool Spend Proposal Amounts 15
- M-03 Pool with Identical Tokens After Wrapping 15
- M-04 Halted Emission Module Indirectly Disables Multiple Core Modules 16
- M-05 Zero Amount Delta in Swap Callback 16
- M-06 Reposition Lacks Deadline Enforcement 17
- M-07 Swap Callback Invoked Before Token Transfer Breaks Flash-Swap Invariant 17
- M-08 Improper Write Authorization to Implementation Realm 18
- M-09 Incorrect Overflow Detection in MulOverflow 18

Low Severity 19
- L-01 Swap Callback Could Point to Arbitrary Realm 19
- L-02 Swap Math Allows Fee Parameter That Causes Division by Zero 20
- L-03 Absence of Halt Checks in Hook-Setting Functions 20
- L-04 Missing Incentive Start Time Check in isActive 21
- L-05 Dry Swap Exact-Output Check Does Not Match Router Tolerance 21
- L-06 Potential Reentrancy Risk in CollectReward 22

L-07 DecreaseLiquidity Slippage Check Uses Owned Amounts Instead of Actually Received Amounts 22
L-08 Minor Deviation in TWAP Rounding Direction for Tick Calculation 23

Notes & Additional Information 23
N-01 Incorrect Error Messages 23
N-02 Incorrect Comments 24
N-03 Duplicate And Unused Code 25
N-04 External Incentive Archiving Can Be Blocked by Non-Unstaking Positions 26

Conclusion 27

Appendix 29
Issue Classification 29

Summary

Type	DeFi	Total Issues	24 (24 resolved)
Timeline	From 2025-11-10 To 2025-12-05	Critical Severity Issues	2 (2 resolved)
Languages	Gnolang	High Severity Issues	1 (1 resolved)
		Medium Severity Issues	9 (9 resolved)
		Low Severity Issues	8 (8 resolved)
		Notes & Additional Information	4 (4 resolved)

Scope

OpenZeppelin audited the gnoswap-labs/gnoswap repository at commit 10f01f0.

In scope were the following files:

```
contract
├── p/
│   ├── gnsmath/
│   │   ├── sqrt_price_math.gno
│   │   └── swap_math.gno
│   ├── rbac/
│   │   ├── ownable.gno
│   │   ├── rbac.gno
│   │   └── role.gno
│   ├── store/
│   │   └── kv_store.gno
│   ├── uint256/
│   │   └── fullmath.gno
│   └── version_manager/
│       └── version_manager.gno
└── r/gnoswap/
    ├── access/
    │   └── access.gno
    ├── common/
    │   ├── coins.gno
    │   └── tick_math.gno
    ├── pool/
    │   │   ├── v1/
    │   │   ├── liquidity_math.gno
    │   │   ├── manager.gno
    │   │   ├── pool.gno
    │   │   ├── position.gno
    │   │   ├── swap.gno
    │   │   └── transfer.gno
    │   ├── proxy.gno
    │   ├── pool.gno
    │   ├── state.gno
    │   ├── store.gno
    │   └── upgrade.gno
    ├── position/
    │   │   ├── v1
    │   │   ├── burn.gno
    │   │   ├── liquidity_management.gno
    │   │   ├── manager.gno
    │   │   ├── mint.gno
    │   │   ├── native_token.gno
    │   │   ├── position.gno
    │   │   └── reposition.gno
```

```
    │   ├── position.gno
    │   ├── proxy.gno
    │   ├── state.gno
    │   ├── store.gno
    │   └── upgrade.gno
    ├── rbac/
    │   ├── ownership.gno
    │   └── rbac.gno
    ├── router/
    │   │   ├── v1
    │   │   ├── base.gno
    │   │   ├── exact_in.gno
    │   │   ├── exact_out.gno
    │   │   ├── router.gno
    │   │   ├── swap_inner.gno
    │   │   ├── swap_multi.gno
    │   │   └── swap_single.gno
    │   ├── proxy.gno
    │   ├── state.gno
    │   ├── store.gno
    │   └── upgrade.gno
    └── staker/
        │   ├── v1
        │   ├── calculate_pool_position_reward.gno
        │   ├── external_incentive.gno
        │   ├── incentive_id.gno
        │   ├── mint_stake.gno
        │   ├── reward_calculation_incentives.gno
        │   ├── reward_calculation_pool_tier.gno
        │   ├── reward_calculation_pool.gno
        │   ├── reward_calculation_tick.gno
        │   ├── reward_calculation_types.gno
        │   ├── reward_calculation_warmup.gno
        │   ├── reward_calculation.gno
        │   ├── staker.gno
        │   ├── type.gno
        │   └── wrap_unwrap.gno
        ├── accessor.gno
        ├── deposit.gno
        ├── pool.gno
        ├── proxy.gno
        ├── state.gno
        ├── store.gno
        └── upgrade.gno
```

System Overview

GnoSwap is a concentrated-liquidity Automated Market Maker (AMM) that is built on the Gno.land platform. Similar to Uniswap V3, it allows liquidity providers to allocate liquidity within selected price ranges, greatly improving capital efficiency. GnoSwap uses a four-tier fee system: 0.01%, 0.05%, 0.3%, and 1%, each with different tick spacings to accommodate stable or volatile asset pairs. Creating a pool requires a 100 GNS fee to prevent spam.

GnoSwap's architecture tightly couples its pool and position realms:

- The pool realm handles swaps, liquidity operations, and fee accounting.
- The position realm manages non-transferable NFT-based positions that represent liquidity, enabling users to mint, modify, or collect fees from their positions.

Swaps can be executed directly, by calling the pool, or via a router that supports single- and multi-hop routing (up to three hops), split routes, and native token (GNOT) wrapping/unwrapping. Slippage protection ensures safe execution.

The protocol offers a two-layer staking incentive system:

- Internal GNS rewards, distributed to governance-approved pools across tiers receiving 50%, 30%, and 20% of emissions.
- External rewards, created by users who deposit reward tokens and 1000 GNS collateral for custom incentive campaigns.

Rewards are subject to a warmup schedule that gradually increases reward rates over time to discourage short-term liquidity. Undistributed rewards redirect to the community pool (internal incentives) or back to the incentive creator (external incentives). A hook-based mechanism keeps the staker realm synchronized with real-time events, including swaps and reward distribution changes, to ensure accurate reward allocation.

This review is an extended audit of a previous audit of the AMM core modules, focusing on major changes in the in-scope files. Files identified as critical to user logic, fund flows, native token handling, and role-based access control were also included in the scope. This audit also covered in-scope fixes for unresolved issues in the previous audit. The following sections provide a brief overview of these changes. For readers interested in a deeper exploration of the underlying design and previous findings, please refer to our earlier audit report.

Upgradeability Architecture

The upgradeability architecture of GnoSwap has greatly improved since the last audit. It now enables the protocol to evolve its business logic while maintaining a stable public interface and centralized data storage. It is designed to enhance operational stability, and the GnoSwap team does not have any plans to add new functions or features that will change the public interface. The system is organized around a proxy pattern that decouples three primary concerns:

- **Proxy Layer (Public Interface)**: Stable entry points exposed to users and integrators.
- **Implementation Layer (Versioned Business Logic)**: Per-domain contracts that encode the functional behavior of the protocol and can be upgraded over time.
- **Storage Layer (Centralized Data Management)**: Stable storage infrastructure that provides type-agnostic key–value storage and access control.

Upgradeability is implemented per "domain" (e.g., pool, position, router, staker, etc.). Each domain has its own proxy, implementation, and state contracts, while all domains build on a shared set of utility contracts for storage, access control, role management, and emergency halt functionality. Versioned implementations can be introduced and activated without migrating data, as all versions read from and write to the corresponding domain's storage.

Upgrades in GnoSwap follow a controlled, per-domain workflow. New implementations register an initializer that describes how to construct the domain’s logic using its persistent store. Administrators may then authorize an upgrade through dedicated upgrade contracts that validate permissions and confirm that the requested implementation has been registered. Once activated, the proxy updates its implementation pointer, immediately routing all subsequent calls to the new logic.

Since domain implementations operate on the same centralized storage and share a stable interface, upgrades occur with zero downtime, no data migration, and immediate rollback capability. This workflow provides a safe and predictable mechanism for iteratively enhancing protocol functionality.

Swap Callback

GnoSwap introduces a new swap callback mechanism that extends the existing swap flow with an additional layer of interaction between the pool contract and calling applications. This callback is designed to give callers explicit control over how tokens are sourced or moved during a swap, enabling more flexible integrations while preserving the security properties of the pool’s core logic.

The updated swap architecture adds a callback step into the execution path between transferring tokens out and receiving tokens in. While the pool continues to determine swap pricing, state transitions, and token deltas, it now invokes a user-supplied callback contract to finalize the required token movements. This pattern mirrors well-established designs in other AMM ecosystems, allowing external applications, such as routers, aggregators, or vaults, to dynamically provide or pull tokens during the swap.

Security Model and Trust Assumptions

This audit depends on several layers of trust assumptions, spanning the broader Gno ecosystem infrastructure as well as protocol-level design choices. The assumptions documented in the prior audit remain applicable and are not repeated here. The sections below exclusively focus on newly introduced or modified considerations.

Upgrade Authorities

The protocol grants administrative entities the ability to register new implementations and activate them through the upgrade contracts. These actors are considered fully trusted within the system's model and can:

- deploy arbitrary code as future implementations
- replace active implementations at any time
- modify the execution flow through interface-compliant but otherwise unconstrained logic

The upgrade authority is effectively equivalent to protocol ownership. Any compromise, misconfiguration, or malicious behavior at this layer can alter protocol semantics, bypass validations, or drain funds unless additional social or procedural safeguards are in place.

Upgrade to the Same Version

Currently, the version manager module allows administrative entities to upgrade the implementation to the same version, which will trigger the initialization process again. This provides a mechanism to reinitialize storage keys that may not have been properly initialized

before. It is assumed that the related implementation's initializer can handle such recovery scenarios correctly.

Permission Reset During Implementation Changes

When `version_manager.ChangeImplementation` activates a new implementation, it revokes write permissions from all previously authorized callers, ensuring that only the newly selected implementation retains modification rights. Domains that rely on additional writable modules must therefore manually restore their permissions during the upgrade process. For example, the staker module's `updateImplementation` re-registers write access to its emission module via `initRegisterWriteableContract`.

This design enforces strict authorization hygiene but introduces operational risk: incomplete or incorrect permission restoration can leave dependent modules unable to write to storage. The system thus relies on upgrade procedures and domain implementers to correctly reinitialize required permissions after each change.

Router Fee

Unlike Uniswap V3, the router module charges an extra fee on the output tokens of each swap. To avoid the fee, users can perform swaps through their own contracts. This is considered intended behavior.

Privileged Roles

The privilege structure of the admin role and governance remains unchanged from the previous audit. These roles continue to share extensive overlapping authorities (listed below), providing them with near-complete control over protocol economics and operational parameters.

- **Fee Management**: Authority to modify all protocol fee parameters without effective limits, including withdrawal fees on collected swap fees, unstaking fees on staking rewards, protocol fees, and router fees.
- **Incentive Pool Management**: Authority to manage the tiered pool system by adding or removing pools from tiers, directly controlling which pools receive GNS emission rewards and their distribution percentages.

- **Reward Schedule Control**: Authority to modify the warmup schedule that determines reward vesting percentages and timeframes.
- **Emission Distribution**: Authority to control the allocation percentages of GNS emissions between stakers, developer operations, the community pool, and governance stakers.
- **Protocol Fee Collection**: Authority to withdraw accumulated protocol fees from swap operations to arbitrary addresses without any enforced limits.
- **Access-Control Management**: Authority to register new roles, update role addresses, and remove non-system roles through the RBAC system.
- **Swap Whitelist Management**: Authority to control which addresses are permitted to directly call the swap function.
- **Operational Halting**: Authority to alter halt levels to pause protocol operations at a granular level, including halting all withdrawals.
- **Economic Parameter Control**: Authority to modify the pool creation fee and external incentive collateral requirements.
- **Token Registration**: Authority to determine which tokens are permitted to operate within the AMM through the token registration system.
- **Upgradeability**: Authority to upgrade realm versions via the RBAC management system.

Critical Severity

C-01 Router Swap Callback Caller Is Not Verified

The router module implements the pool's swap callback interface through the `SwapCallback` function. This function is intended to only be invoked by the canonical pool realm as part of the swap process, at which point the router transfers the required input tokens back to the pool. However, the router does not verify that the caller of `SwapCallback` is the legitimate implementation realm.

Due to no realm or address check being performed, any malicious user can invoke this callback directly, supplying forged payer and amount parameters. When the router processes these arguments, it would transfer tokens from arbitrary users with previous unspent approvals to the router realm, resulting in loss of funds.

Consider implementing strict caller verification within `SwapCallback` which confirms that the caller is indeed the canonical realm before processing the callback.

Update: *Resolved in pull request #1005 at commit b6ef5c8 and pull request #1075 at commit e0b70dc. The team stated:*

> *Added `assertIsRouterV1()` function to verify that the caller's address matches the router v1 contract address. Introduced `routerV1Addr` constant in consts.gno to store the canonical router v1 package address. Implemented caller verification and halt state check in `SwapCallback` before proceeding with token transfer logic. Added `access.AssertIsPool(caller)` assertion to the swap callback function to ensure that only registered pool contracts can call the callback. Updated the documentation and comments in the router implementation.*

C-02 Swap Reentrancy Lock Is Ineffective

The Swap function in the pool attempts to protect against reentrancy by using the `Slot0.unlocked` field as a lock. The intended flow is as follows:

1. Read `Slot0`.
2. Set `unlocked = false` before executing external callbacks.
3. Restore `unlocked = true` when finished.

However, the lock never actually takes effect because `Slot0` returns a copy of the struct instead of a pointer. When `SetUnlocked(false)` is called on this value, it only mutates the local copy, not the stored pool state. Despite this, the external callback (`swapCallback` via `safeSwapCallback`) is still invoked afterward. Since the stored pool state always has `unlocked == true`, reentrant calls into `Swap` from inside the callback are not blocked. In addition, `safeSwapCallback` performs a balance check after the callback to ensure the pool has received the required tokens. Since reentrancy is possible, an attacker can, during the callback, invoke other pool/position logic such as `IncreaseLiquidity` to manipulate balances in a way that bypasses the intended repayment checks.

Consider ensuring that the reentrancy lock correctly protects `Swap` by making `Slot0` return a pointer (or equivalent reference type) so that `SetUnlocked(false)` and subsequent writes update the persisted state.

Update: *Resolved in pull request #1006 at commit 1acb0c3. The team stated:*

> *Added `pool.SetSlot0(slot0Start)` after setting the unlock flag to false at swap start, ensuring the reentrancy lock is properly reflected in the pool's state before executing the swap. Changed the unlock logic in the defer function to fetch the current pool state via `pool.Slot0()` instead of using the local variable `slot0Start`. The lock is now released by setting unlock to `true` on the fetched state and applying it back via `pool.SetSlot0(slot0End)`, ensuring the lock operates on the actual current state rather than a stale copy.*

High Severity

H-01 Incorrect Iteration Control in `IsExternallyIncentivizedPool` Stops Staking

In the staking module, `IsExternallyIncentivizedPool` is used by `poolHasIncentives`, which in turn guards the staking in `StakeToken` by requiring a pool to be internally or externally incentivized before accepting new deposits. The iteration callback inside `IsExternallyIncentivizedPool` returns `true` when an incentive is *inactive*, which stops traversal of the incentives tree. As a result, if an inactive incentive appears earlier in the tree than an active one, the function will incorrectly report that the pool has no active external incentives, causing `poolHasIncentives` to reject staking into pools that do in fact have at least one active external incentive.

Consider inverting the loop control in `IsExternallyIncentivizedPool` so that it continues iterating over inactive incentives and breaks only when an active incentive is found (for example, by returning `false` for inactive entries and `true` once `isActive` returns true).

Update: *Resolved in pull request #1037 at commit 8890c30. The team stated:*

> *Inverted the return values in the iteration callback of `IsExternallyIncentivizedPool`. The callback now returns `false` when an active incentive is found (to stop traversal after setting `hasActive = true`), and returns `true` for inactive entries (to continue iterating). This ensures the function correctly identifies pools with active external incentives even when inactive incentives appear earlier in the tree.*

Medium Severity

M-01 None Permission Can Be Escalated to Read After Upgrade

The `store` package provides a domain-specific key-value storage system with permission-based access control for realms. The package defines three permission levels:

- `None` (0): No access to the store.
- `ReadOnly` (1): Can read data but cannot modify.
- `Write` (2): Can both read and write data.

To update the permission of an existing authorized caller, realms can call `UpdateAuthorizedCaller`, which accepts a `Permission` parameter that can override the previous permission. Notably, the package also provides a `RemoveAuthorizedCaller` function to remove an authorized caller, which is effectively the same as updating the permission to `None`. In the version manager package, when performing a hot-swap to a different version implementation, it will replace all authorized permissions to read-only except for the active version. However, if there is a None permission, it will be escalated to `ReadOnly`, which may lead to unexpected results, such as accessing incorrect data in getter functions.

Consider only allowing `ReadOnly` and `Write` as parameter values in `UpdateAuthorizedCaller`. This would force realms to use `RemoveAuthorizedCaller` if they want to remove/reset the permission.

Update: *Resolved in pull request #1043 at commit 3829ce5.*

M-02 Inadequate Validation of Community Pool Spend Proposal Amounts

The `validateCommunityPoolSpend` function validates community pool spend proposal data. Specifically, it verifies the recipient address, the token path, and the amount. However, when confirming that the amount is positive, the function only ensures that it is not zero, allowing users to construct proposals with negative token transfer amounts. This creates ambiguity in proposal semantics and may enable unintended behavior depending on how downstream modules interpret negative values. Although this issue does not directly enable theft or unauthorized transfers, it introduces correctness risks in governance-driven fund movements.

Consider strengthening the validation logic by ensuring that the community pool spend amount is strictly positive.

Note: The governance module is out of scope for this audit.

Update: *Resolved in pull request #1042 at commit e135305.*

M-03 Pool with Identical Tokens After Wrapping

In the `CreatePool` function, the call to `assertIsNotEqualsTokens` correctly verifies that the two tokens supplied by the user are different at submission time. However, after this check, the function calls `updateWithWrapping`, which automatically wraps the native token GNOT into its wrapped version `WRAPPED_WUGNOT`. No subsequent verification is performed to ensure that the token pair remains distinct after this wrapping step.

As a result, a user can create a pool using the `wugnot/gnot` pair and produce a pool with identical tokens (`wugnot/wugnot`). An important invariant of every AMM pool is that its two tokens must be distinct. Multiple parts of the protocol rely on this assumption. Downstream logic does check that `token0 != token1` and would revert accordingly, leaving such a pool effectively unusable.

Consider ensuring that token distinctness is validated after wrapping by placing the `assertIsNotEqualsTokens` check after the `updateWithWrapping` step.

Update: *Resolved in pull request #1047 at commit a6c5092.*

M-04 Halted Emission Module Indirectly Disables Multiple Core Modules

The `halt` module is designed to provide granular control over protocol operations, enabling both emergency responses and a beta safety mode. It allows different modules and operations to be halted independently. However, many core modules invoke `emission.MintAndDistributeGns` as part of their normal flows in order to distribute newly minted GNS tokens.

If the emission module is halted, all these other modules will panic or revert when they attempt to mint and distribute GNS. As a result, they become effectively inoperable, despite their own halt states potentially allowing operation. This behavior significantly reduces the effective granularity of the halt system, since halting emission implicitly halts multiple unrelated modules.

Consider wrapping calls to `emission.MintAndDistributeGns` in other modules within a `try-catch`-like block (or equivalent error-handling mechanism). This is so that the halting emission does not unintentionally block the rest of the protocol. Alternatively, consider redesigning the `MintAndDistributeGns` function without panic.

Update: *Resolved in pull request #1051 at commit 9dfd7da. The team stated:*

> *Modified the existing MintAndDistributeGns function to return (int64, bool) instead of causing a panic in the emission halt check section. This allows other contracts to operate normally regardless of the emission contract's state. The bool indicates whether the state is active.*

M-05 Zero Amount Delta in Swap Callback

The swap callback contains two parameters, `amount0Delta` and `amount1Delta`, both of which are string values. Usually, these two deltas represent the tokens that were sent (negative) or must be received (positive) by the pool by the end of the swap. However, the current implementation only sets the positive delta and does not set the negative delta. In

addition, there is no documentation about the usage of these two parameters. This could result in the following problems:

- The callback cannot (easily) get the tokens swapped out, especially when the recipient is another entity
- Incorrect assumptions for custom callbacks that could lead to denial of service
- Other issues

Consider following the standard practice by populating both `amount0Delta` and `amount1Delta` with proper positive/negative values, matching the established Uniswap V3 convention.

Update: *Resolved in pull request #1009 at commit 64fcb42 and pull request #1075 at commit e0b70dc.*

M-06 Reposition Lacks Deadline Enforcement

The `Reposition` function performs liquidity and position changing updates without enforcing any deadline, unlike `Mint`, `IncreaseLiquidity`, and `DecreaseLiquidity`, which all validate a user-specified deadline via `assertIsNotExpired`. As a result, a signed `Reposition` transaction can be executed later, potentially after market movements, leading to stale execution, unexpected liquidity placement, and increased exposure to MEV activities. This breaks the assumption that all major position-altering actions are time-bounded by the user.

Consider adding a deadline parameter to `Reposition` and invoking the same `assertIsNotExpired` check used by the other state-changing functions.

Update: *Resolved in pull request #1052 at commit c209f8b.*

M-07 Swap Callback Invoked Before Token Transfer Breaks Flash-Swap Invariant

The `Swap` function invokes `safeSwapCallback` before transferring the output tokens to the swap recipient. This ordering violates a key invariant of flash-swap–compatible designs, which is assumed in both the pool's own swap logic and the router's callback logic.

Since the callback is executed before the recipient receives the swapped token, users cannot rely on those tokens inside the callback. This prevents legitimate use cases such as performing

arbitrage or chained operations using the output amount before repaying the required input amount.

Consider reordering the operations so that the swapped token is transferred before invoking the callback. This would ensure compatibility with standard flash-swap expectations and prevent functional limitations for integrators.

Update: *Resolved in pull request #1008 at commit f59ded1.*

M-08 Improper Write Authorization to Implementation Realm

The proxy's version management system is implemented in the `version_manager` package. When a new active implementation is registered or upgraded, this package grants storage write permission to the implementation realm. However, because calls from the proxy realm to the implementation realm do not change the realm context's `runtime.CurrentRealm()`, this grant is unnecessary. The proxy already possesses the required write permission to the storage layer. More importantly, this improper authorization could lead to unexpected behavior if the implementation realm is invoked externally and modifies the store.

Consider preventing implementation realms from receiving write permission.

Update: *Resolved in pull request #1048 at commit da4483d. The team stated:*

> *Removed all storage write permission grants to implementation realms in the version manager. The register and updateImpl functions no longer authorize implementation realms, since proxy-to-implementation calls do not change runtime.CurrentRealm() and the proxy already possesses the required write permission. This prevents potential unexpected behavior where implementation realms could modify the store when invoked externally.*

M-09 Incorrect Overflow Detection in `MulOverflow`

The `MulOverflow` helper returns both the product and an overflow flag. This routine is used to safely detect multiplication overflows when working near the limits of the signed 256-bit range.

The issue is that `MulOverflow` computes the product using absolute magnitudes but applies an overflow check that unconditionally treats any negative intermediate as overflow (256th bit set), without taking into account whether the final result is a positive or negative value. As a consequence, certain valid negative products whose magnitude is exactly `-2^255` (E.g., `-2^254 * 2`), which is the minimal negative value, are incorrectly flagged as overflow.

Consider updating the overflow detection logic to handle such edge cases.

Note: The `int256` package is out-of-scope of this audit, and the issue was introduced after the audit ended.

Update: *Resolved in pull request #1061 at commit c597637. The team stated:*

> *In `MulOverflow`, when the 256th bit is set, an additional check was added to handle it as valid only if the value is -2^255. Related tests were added, and unit tests for the `Mul` function were also added for additional verification.*

Low Severity

L-01 Swap Callback Could Point to Arbitrary Realm

During the swap process, users pass a callback function of type `func(currentRealm, amount0Delta, amount1Delta string) error`. This function is invoked inside the pool module's `safeSwapCallback` logic to notify the user of the swap deltas. However, the pool module does not validate that the callback belongs to the caller's realm (`runtime.PreviousRealm()`). As a result, a malicious user can invoke any arbitrary or privileged callback. (e.g., one callable only by the pool module), enabling the pool module to execute it and potentially resulting in unintended consequences.

Currently, there is no such pool callable privileged logic, nor does the development team have any plans to introduce it in the future. This still leaves the module exposed if such logic were to be added at any point.

Consider adding a verification step to confirm that the callback is associated with the previous realm. Rejecting swaps where the callback function is not part of the caller's realm.

Update: *Resolved in pull request #1059 at commit a0148a7. The team stated:*

> *We added a `CallbackMarker` type parameter to the swap callback signature. This enforces type-system level validation, so existing privileged functions with different signatures cannot be passed as callbacks. Any callback must now explicitly include the marker parameter, preventing arbitrary function injection.*

L-02 Swap Math Allows Fee Parameter That Causes Division by Zero

The `swap_math.gno` package is publicly accessible and may be invoked by external users or realms. Within the `SwapMathComputeSwapStep` function, the input validation logic permits the fee parameter `feePips` to be equal to the fee denominator (`1_000_000`). The problematic case occurs when `feePips` equals `1_000_000` and the execution takes the code path that computes the fee via `MulDivRoundingUp(amountIn, feePips, denominator - feePips)`. The function will attempt to divide by zero, causing a panic.

Consider tightening the input validation (`feePips < denominator`), or alternatively adding an explicit special-case branch for it.

Update: *Resolved in pull request #1045 at commit 23c7584.*

L-03 Absence of Halt Checks in Hook-Setting Functions

The `SetTickCrossHook`, `SetSwapStartHook`, and `SetSwapEndHook` functions are external state-changing functions in the pool module that can be invoked by the staker module. Most other state-changing functions in the pool module enforce the halt mechanism by calling `halt.AssertIsNotHaltedPool()` (or equivalent) to ensure that operations cannot be performed while the pool is halted. However, these three setter functions do not perform such a check. As a result, hook configuration can still be modified even when the pool is supposed to be halted.

Consider adding a call to `halt.AssertIsNotHaltedPool()` at the beginning of each of the aforementioned functions.

Update: *Resolved in pull request #1046 at commit 22dc66f. The team stated:*

> *Added `halt.AssertIsNotHaltedPool()` to the `SetTickCrossHook`, `SetSwapStartHook`, and `SetSwapEndHook` functions to check the pool's halt state. I also added tests for the halt state behavior.*

L-04 Missing Incentive Start Time Check in `isActive`

During staking, `StakeToken` calls `poolHasIncentives` to ensure that the target pool is either internally or externally incentivized, and `poolHasIncentives` in turn relies on `IsExternallyIncentivizedPool`, which uses `isActive` to decide whether any external incentive is active. However, `isActive` currently returns `true` whenever `currentTimestamp < EndTimestamp()` and does not check `StartTimestamp()`. So, an incentive that starts far in the future but has not yet ended is treated as "active" during staking checks. As a result, a malicious user can create an external incentive with the minimum allowed reward amount and a start time 100 years in the future, causing the protocol to regard the pool as having an active external incentive even though it will remain effectively inactive for the foreseeable lifetime of the system.

Consider tightening the definition of activity by updating `isActive` (or the logic in `IsExternallyIncentivizedPool`) to require both `currentTimestamp >= StartTimestamp()` and `currentTimestamp < EndTimestamp()`, or ensuring the incentive will start in a limited time.

Update: *Resolved in pull request #1041 at commit 6451b9a. The team stated:*

> *We have changed the conditions of the `isActive` function. Please note that after internal review, the behavior of the `IsExternallyIncentivizedPool` function was determined to be intentional, so no separate handling was applied.*

L-05 Dry Swap Exact-Output Check Does Not Match Router Tolerance

In the router module, for exact-output swaps, the final output amount (after fee deduction) must match the requested amount, with a tolerance of up to `swapCount` units to account for rounding. However, in the `DrySwapRoute` function, which simulates a route without executing it, the corresponding check does not apply the same tolerance rule. As a result, the dry run may reject swap routes that the actual router would accept, marking valid exact-output swaps as invalid in simulation. This creates a discrepancy between simulated behavior and actual execution behavior.

Consider revising the dry swap logic to align it with the actual router behavior.

Update: *Resolved in pull request #1053 at commit a2067d2.*

L-06 Potential Reentrancy Risk in `CollectReward`

In the staking module, `CollectReward` computes internal and external rewards for a position based on the position's last-collect timestamps, then invokes external calls `SafeGRC20Transfer` to transfer rewards to users, before updating the incentive's last-collect times via `updateExternalRewardLastCollectTime`. If a reward token were able to re-enter `CollectReward` (e.g., via a hook during token transfer), the re-entrant invocation would use the old timestamps, leading to incorrect reward calculations.

Consider moving all external token transfers to occur after updating the deposit and incentive state, thus following the Checks-Effects-Interactions pattern.

Update: *Resolved in pull request #1055 at commit 40fc802.*

L-07 `DecreaseLiquidity` Slippage Check Uses Owned Amounts Instead of Actually Received Amounts

The external `DecreaseLiquidity` entry point accepts `amount0MinStr`/`amount1MinStr` slippage parameters and passes them into the internal `decreaseLiquidity`, which calls pool `Burn` to get `burn0`/`burn1` (tokens owed for the removed liquidity), and then checks slippage via `isSlippageExceeded` against those amounts. Then it invokes pool `Collect`, and later transfers `collect0`/`collect1` to the user. However, `collect0/collect1` from `Collect` could be less than `burn0`/`burn1` if constrained by pool balances or other limits. In such edge cases, the transaction can succeed while the caller receives less than the slippage minimum, especially when the user tries to withdraw all the liquidity. In short, the slippage requirement is effectively enforced on "amounts owed" rather than on "amounts actually received."

Consider moving the slippage check to compare `collect0`/`collect1` against the user's `amount0Min`/`amount1Min`, thereby guaranteeing the minimum amount of tokens is actually transferred to the user.

Update: *Resolved in pull request #1050 at commit 6019e88.*

L-08 Minor Deviation in TWAP Rounding Direction for Tick Calculation

The time-weighted average price (TWAP) is computed in the oracle via `getTWAP`. Currently, the average tick is computed as the difference of cumulative ticks `tickDelta` divided by `secondsAgo`, which truncates toward zero instead of flooring toward negative infinity as in Uniswap V3. This means that for negative `tickDelta` values that are not exactly divisible by `secondsAgo`, the TWAP tick is one tick less negative than Uniswap's implementation.

Consider aligning the rounding semantics with Uniswap by applying an explicit adjustment after the division.

Note: The TWAP oracle is out of scope for this audit.

Update: *Resolved in pull request #1040 at commit 758360f.*

Notes & Additional Information

N-01 Incorrect Error Messages

The type-casting helper functions in `contract/p/gnoswap/store/utils.gno` return misleading error messages that invert the source and destination types, potentially confusing operators and integrators when diagnosing failed casts.

The following incorrect error messages were identified:

- `CastToInt64` formats its error detail as `"cast int64 to %T"`, whereas the actual failed operation is casting from the runtime type `%T` to `int64`.
- `CastToUint64` formats its error detail as `"cast uint64 to %T"` instead of indicating a cast from `%T` to `uint64`.
- `CastToBool` formats its error detail as `"cast bool to %T"` instead of indicating a cast from `%T` to `bool`.
- `CastToString` formats its error detail as `"cast string to %T"` instead of indicating a cast from `%T` to `string`.
- `CastToTree` formats its error detail as `"cast *avl.Tree to %T"` instead of indicating a cast from `%T` to `*avl.Tree`.

- `CastToAddress` formats its error detail as `"cast address to %T"` instead of indicating a cast from `%T` to `address`.

Consider updating the aforementioned error messages so that they consistently report failed casts in the form `"cast %T to <target type>"`, and reviewing similar helper functions across the codebase to ensure that all error messages accurately describe the attempted operation.

Update: *Resolved in pull request #1044 at commit 3a8e3bc.*

N-02 Incorrect Comments

Several comments and docstrings in the pool, position, and staker modules are incorrect or vague, which can mislead integrators and reviewers about actual behavior, invariants, and return values. This issue lists the identified discrepancies so that documentation can be aligned with the current code.

- The documentation for `safeTransfer` states that the `amount` parameter "must be negative" and that the function validates a negative amount. However, the implementation uses an unsigned `*u256.Uint` and treats `amount` as a positive quantity that is subtracted from the pool balances. As such, the comments should describe `amount` as a positive transfer amount instead of a negative delta.
- The doc comment for `Swap` describes a `data` parameter being passed through to the swap callback, but the current function signature and the `swapCallback` type contain no such `data` parameter and no payload is propagated to the callback. Thus, either the parameter needs to be reintroduced or the comment updated to remove references to `data`.
- The return-value documentation for `IncreaseLiquidity` states that `liquidity` is the "Total liquidity after increase". However, the function actually returns the incremental liquidity added (the delta) alongside the existing `positionId`, token amounts, and `poolPath`.
- The comment for `getPositionKey` claims that it "Ensures unique positions per owner and price range". However, the key is only derived from `positionPackagePath`, `tickLower`, and `tickUpper` (no owner component). So, uniqueness is per package or price range rather than per owner.
- The inline comment in `modifyWarmup` says "Handle negative duration - set to math.MaxInt64". However, the implementation panics when `timeDuration < 0` instead of clamping to `math.MaxInt64`. As such, the comment should be updated to reflect the actual error behavior.

- The docstring for `safeTransferFrom` states that the function validates and converts the amount to `uint64` via `safeConvertToUint64`. However, the implementation converts to `int64` using `safeConvertToInt64` and performs all arithmetic and balance checks in terms of signed 64-bit integers.
- The parameter documentation for `Mint` still refers to a `recipient` parameter. However, the current signature no longer accepts `recipient` and instead uses `positionCaller` as the address that ultimately provides tokens. As such, the comments should be updated to either remove or rewrite the obsolete `recipient` description.
- The parameter list in the `SwapWithCallback` comment block still refers to `tokenIn` and `tokenOut`. However, the actual `Swap` signature now uses `token0Path` and `token1Path`. Thus, the documentation should be updated to match the current parameter names and semantics.
- The return-value comments for `Collect` state that `amount0` and `amount1` are "actually collected (after fees)". However, the function returns transferred amount without applying withdrawal or protocol fees. Since fee handling is performed elsewhere, the "after fees" wording here is misleading.

Consider updating the aforementioned comments and docstrings so that they accurately reflect the current behavior and types.

Update: *Resolved in pull request #1049 at commit b7439b9.*

N-03 Duplicate And Unused Code

The codebase contains several instances of duplicate or unused logic across the staking, pool, router, position, access, and RBAC realms. These patterns increase maintenance overhead, obscure the effective behavior of the contracts, and can make it harder to reason about reward accounting and error handling.

- At the end of the `CollectReward` function there are two calls to `SetTotalEmissionSent`, resulting in redundant state updates (one of which ignores the returned error).
- `assertPoolMustExist` is not invoked by production code and is only exercised in tests.
- The `poolTier` parameter of `processUnclaimableReward` is unused, indicating a vestigial or incomplete refactoring.
- The position realm constant `GNOT_DENOM` is defined but never read within the package.
- `GNS_ADDR` in the RBAC realm is defined but not referenced elsewhere.

- The router-level `maxApprove` constant is only used in tests and not in production logic.
- Several errors across all realms and directories are unused or only referenced in tests. This suggests that they may be redundant and should be re-examined.

Consider either removing or refactoring any unused or duplicate code to eliminate redundant logic while updating the test suite after any removals.

Update: *Resolved in pull request #1054 at commit 626514d.*

N-04 External Incentive Archiving Can Be Blocked by Non-Unstaking Positions

The archiving of external incentives is triggered both at the end of `EndExternalIncentive` and during `applyUnStake`. However, both paths only call `archiveEndedExternalIncentive` when `ExternalIncentiveResolver.IsFinished` evaluates to `true`. This predicate requires the incentive to be marked as refunded and for its recorded distributed reward amount to reach the configured total reward amount, but the distributed amount is only increased when positions are explicitly unstaked via `applyUnStake`. As a result, a user who never unstakes can prevent the `IsFinished` condition from being satisfied, preventing the external incentive from being archived.

Consider revisiting the completion and archiving criteria for external incentives so they cannot be blocked by non-unstaking positions.

Update: *Resolved in pull request #1060 at commit 0f5d968. The team introduced a new tracking method for external incentives and stated:*

> *The archive system is no longer needed, as deposit-level tracking provides the same guarantees (preventing double-claims) while offering better performance and simpler state management.*

Conclusion

This extended audit covers GnoSwap's core modules, especially the improvements made to the upgradability framework since the previous audit, along with the addition of the swap callback mechanism. These updates represent meaningful progress toward a more modular and maintainable system, particularly a clearer separation between proxy, implementation, and storage layers and the more predictable upgrade workflow across domains.

Overall, notable improvements in the code quality of the core modules and their associated test cases were observed. Most issues identified in this review arise from newly introduced logic, particularly the major architectural adjustments implemented during the fix review of the prior audit. That said, several out-of-scope issues were identified across both engagements, indicating that non-audited modules are likely to present potential vulnerabilities or inconsistencies. Increasing internal review processes and expanding audit coverage would materially improve the likelihood of surfacing issues that fall outside the current scope.

During this assessment, 2 critical-, 1 high-, 9 medium-, and several low-severity issues and notes were identified. While much of the codebase reflects thoughtful design evolution, several components would benefit from strengthened invariants and clearer documentation of the intended behavior. The introduction of architectural enhancements and callback-based swap flows positions GnoSwap for future extensibility. However, it also increases the importance of rigorous internal processes, including testing, documentation, and governance controls.

To further strengthen the protocol's overall quality and deployment readiness, a multi-stage rollout approach is recommended. This could begin with an alpha phase featuring restricted or whitelisted access, enforced transaction caps, and simulated or controlled environments to surface potential edge cases. A subsequent beta phase could involve broader user participation with progressively increased limits, complemented by automated invariant checks and active monitoring to validate system behavior under varied conditions.

In parallel, establishing a bug bounty program early in the rollout would incentivize external security researchers to identify issues that may only emerge under real-world usage. Integrating external reviews for material changes and fixes could further reduce residual risk. Together with data gathered during phased deployment, these measures would provide valuable insights and meaningful assurance, supporting a safe transition toward full mainnet availability.

The GnoSwap team is appreciated for consistently demonstrating strong responsiveness and a collaborative approach, materially contributing to the effectiveness of the audit.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.